

Orchestrator.py

```
# core/orchestrator.py

import logging

import re

from config.settings import RAG_THRESHOLD

from cache.redis_manager import redis_cache

from cache.context_cache import context_cache


logger = logging.getLogger(__name__)


class Orchestrator:

    """Orquestador inteligente que decide entre RAG y CAG"""

    def __init__(self):

        self.rag_threshold = RAG_THRESHOLD

        logger.info(f"✅ Orquestador inicializado con umbral RAG: {self.rag_threshold}")

    def should_use_rag(self, query):

        """

        Decide si usar RAG basándose en la complejidad de la consulta

        """

        if self._is_complex_query(query):

            logger.debug("⚡ Consulta compleja detectada → usar RAG")

            return True

        elif self._is_simple_query(query):

            logger.debug("⚡ Consulta simple detectada → usar CAG")

            return False

        else:
```

```
# Por defecto, usa RAG si no cae en ninguna categoría clara

logger.debug(" ⚡ Consulta ambigua → usar RAG por defecto")

return True
```

```
def _is_simple_query(self, query):
```

```
    """Determina si la consulta es simple (puede ser respondida con documentos cacheados)"""
```

```
    # Consultas muy cortas probablemente sean simples
```

```
    if len(query.split()) <= 3:
```

```
        return True
```

```
    # Patrones de consultas simples
```

```
    simple_patterns = [
```

```
        r"qué es.*",
```

```
        r"quien es.*",
```

```
        r"cómo funciona.*",
```

```
        r"definición de.*",
```

```
        r"explica.*",
```

```
        r"habla sobre.*"
```

```
    ]
```

```
    query_lower = query.lower()
```

```
    for pattern in simple_patterns:
```

```
        if re.match(pattern, query_lower):
```

```
            return True
```

```
    return False
```

```
def _is_complex_query(self, query):
```

```
    """Determina si la consulta es compleja (requiere RAG)"""
```

```
# Consultas largas probablemente sean complejas
```

```
if len(query.split()) > 8:
```

```
    return True
```

```
# Patrones de consultas complejas
```

```
complex_patterns = [
```

```
    r"comparar.*",
```

```
    r"ventajas y desventajas.*",
```

```
    r"pros y contras.*",
```

```
    r"diferencia entre.*",
```

```
    r"ejemplo de.*",
```

```
    r"cómo hacer.*",
```

```
    r"pasos para.*",
```

```
    r"mejor manera de.*",
```

```
    r"qué pasa si.*"
```

```
]
```

```
query_lower = query.lower()
```

```
for pattern in complex_patterns:
```

```
    if re.match(pattern, query_lower):
```

```
        return True
```

```
return False
```

```
def route_query(self, query):
```

```
    """
```

```
    Decide la ruta para la consulta y devuelve el tipo de camino
```

```
    Returns:
```

```
        str: "rag" o "cag"
```

```
"""
```

```
if self.should_use_rag(query):
```

```
    return "rag"
```

```
else:
```

```
    return "cag"
```

```
# Instancia singleton para ser importada
```

```
orchestrator = Orchestrator()
```

cag_engine.py

```
# core/cag_engine.py
```

```
import logging
```

```
from models.llms import azure_llm_client
```

```
from cache.context_cache import context_cache
```

```
from cache.redis_manager import redis_cache
```

```
logger = logging.getLogger(__name__)
```

```
class CAGEngine:
```

```
    """Motor CAG para respuestas eficientes desde caché de contexto"""
```

```
    def __init__(self):
```

```
        logger.info("✅ Motor CAG inicializado")
```

```
    def generate_response(self, query):
```

```
        """Genera una respuesta usando el contexto cacheado"""
```

```
        try:
```

```
            # 1. Buscar en caché de contexto
```

```
            logger.debug(f"🔍 Buscando en caché de contexto: {query}")
```

```
context_results = context_cache.search_in_cache(query)
```

```
if not context_results:
```

```
    return "No tengo información suficiente en mi conocimiento almacenado para responder  
esta pregunta."
```

```
# 2. Construir contexto con los documentos encontrados
```

```
context = self._build_context(context_results)
```

```
# 3. Generar prompt para el LLM
```

```
messages = self._build_messages(query, context)
```

```
# 4. Obtener respuesta del LLM
```

```
logger.debug("🧠 Generando respuesta con Azure OpenAI...")
```

```
response = azure_llm_client.generate_response(messages)
```

```
# 5. Cachear respuesta en Redis
```

```
redis_cache.set_cached_response(query, response)
```

```
logger.debug("✅ Respuesta CAG generada exitosamente")
```

```
return response
```

```
except Exception as e:
```

```
    logger.error(f"❌ Error en el motor CAG: {str(e)}")
```

```
    return "Lo siento, ocurrió un error al procesar tu consulta desde la caché."
```

```
def _build_context(self, context_results):
```

```
    """Construye el contexto a partir de los resultados de caché"""
```

```
    context = ""
```

```
for i, result in enumerate(context_results, 1):  
    context += f"[Documento {i}]: {result['document']}\n\n"
```

```
return context
```

```
def _build_messages(self, query, context):
```

```
    """Construye los mensajes para el LLM con el contexto cacheado"""
```

```
    system_message = """Eres un asistente útil que responde preguntas basándose ÚNICAMENTE  
en el contexto proporcionado desde tu conocimiento almacenado.
```

```
Sigue estas reglas:
```

1. Responde solo con la información del contexto
2. Si la información no está en el contexto, di que no lo sabes
3. Sé conciso y directo
4. Usa el mismo idioma de la pregunta"""

```
    user_message = f"""Contexto de conocimiento almacenado:  
{context}
```

```
Pregunta: {query}
```

```
Por favor, responde basándote solo en el contexto anterior de tu conocimiento almacenado."""
```

```
return [  
    {"role": "system", "content": system_message},  
    {"role": "user", "content": user_message}  
]
```

```
def process_query(self, query):
```

```
"""Método principal para procesar consultas CAG"""
```

```
return self.generate_response(query)
```

```
# Instancia singleton para ser importada
```

```
cag_engine = CAGEngine()
```

main_pdf.py

```
# =====
```

```
# CONFIGURACIÓN SSL GLOBAL - DESACTIVAR VERIFICACIÓN
```

```
# =====
```

```
import os
```

```
import warnings
```

```
# Desactivar advertencias SSL
```

```
warnings.filterwarnings("ignore", message="Unverified HTTPS request")
```

```
# Desactivar completamente la verificación SSL
```

```
os.environ['SSL_CERT_FILE'] = ""
```

```
os.environ['REQUESTS_CA_BUNDLE'] = ""
```

```
os.environ['CURL_CA_BUNDLE'] = ""
```

```
os.environ['PYTHONHTTPSVERIFY'] = "0"
```

```
# Configurar para deshabilitar verificación SSL en todas las librerías
```

```
os.environ['NO_PROXY'] = '*'
```

```
# =====
```

```
# IMPORTACIONES (después de configurar SSL)
```

```
# =====
```

```
import logging
```

```

import ssl

import urllib3

import requests

from fastapi import FastAPI, HTTPException, File, UploadFile

from pydantic import BaseModel

from pathlib import Path

from PyPDF2 import PdfReader


# Desactivar verificación SSL a nivel global

ssl._create_default_https_context = ssl._create_unverified_context

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)


# Configurar sesión global de requests sin verificación SSL

requests_session = requests.Session()

requests_session.verify = False


# =====

# IMPORTACIONES DE MÓDULOS PROPIOS (después de configurar SSL)

# =====

from core.orchestrator import orchestrator

from core.rag_engine import rag_engine

from core.cag_engine import cag_engine

from evaluation.langfuse_monitoring import langfuse_monitor


# =====

# CONFIGURAR LOGGING

# =====

logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)

```

```

# =====

# INICIALIZAR FASTAPI

# =====

app = FastAPI(title="Sistema RAG + CAG", version="1.0.0")


# =====

# MODELOS Pydantic

# =====

class QueryRequest(BaseModel):
    query: str
    user_id: str = "anonymous"


class QueryResponse(BaseModel):
    response: str
    route: str
    source: str


# =====

# ENDPOINTS

# =====

@app.post("/upload_pdf")
async def upload_pdf(file: UploadFile = File(...)):
    """Sube un PDF y lo indexa en Chroma con chunking jerárquico"""
    try:
        file_path = Path("uploaded_docs") / file.filename
        file_path.parent.mkdir(exist_ok=True)

        # Guardar archivo temporal

```

```

with open(file_path, "wb") as f:
    f.write(await file.read())

# Leer texto del PDF
reader = PdfReader(str(file_path))
full_text = ""
for page in reader.pages:
    full_text += page.extract_text() + "\n"

# Indexar en RAG con jerarquía
doc_id = file.filename.replace(".", "_")
rag_engine.ingest_document(doc_id, full_text)

return {"status": "success", "message": f"Documento {file.filename} indexado correctamente."}

except Exception as e:
    logger.error(f"❌ Error subiendo PDF: {e}", exc_info=True)
    raise HTTPException(status_code=500, detail=str(e))

@app.post("/query", response_model=QueryResponse)
async def process_query(request: QueryRequest):
    """Endpoint principal para procesar consultas"""
    try:
        logger.info(f"📄 Consulta recibida: '{request.query}'")

        # 1. Decidir la ruta (RAG vs CAG)
        route = orchestrator.route_query(request.query)
        logger.info(f"🔄 Ruta decidida: {route.upper()}")

```

2. Procesar según la ruta

```
if route == "rag":
```

```
    response = rag_engine.generate_answer(request.query)
```

```
    source = "chroma_db"
```

```
else:
```

```
    response = cag_engine.process_query(request.query)
```

```
    source = "context_cache"
```

3. Monitorear con Langfuse (opcional)

```
try:
```

```
    langfuse_monitor.trace_query(
```

```
        query=request.query,
```

```
        response=response,
```

```
        route=route,
```

```
        user_id=request.user_id
```

```
    )
```

```
except Exception as e:
```

```
    logger.warning(f"⚠️ Error en monitoreo Langfuse: {e}")
```

```
logger.info(f"📦 Respuesta generada via {route.upper()}")
```

```
return QueryResponse(
```

```
    response=response,
```

```
    route=route,
```

```
    source=source
```

```
)
```

```
except Exception as e:
```

```
logger.error(f"❌ Error procesando consulta: {e}")

raise HTTPException(status_code=500, detail="Error interno del servidor")
```

```
@app.get("/health")

async def health_check():

    """Endpoint de health check"""

    return {"status": "healthy", "service": "rag-cag-system"}
```

```
@app.get("/")

async def root():

    """Endpoint raíz"""

    return {

        "message": "Sistema RAG + CAG funcionando",

        "endpoints": {

            "health": "/health",

            "query": "/query (POST)"

        }

    }
```

```
# =====

# MAIN

# =====

if __name__ == "__main__":

    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=9000, ssl_certfile=None)
```

rag_engine.py

```
# core/rag_engine.py

from typing import List, Dict, Any
```

```

from langchain.text_splitter import RecursiveCharacterTextSplitter

import chromadb

import os

from dotenv import load_dotenv

from openai import AzureOpenAI # <-- Cambiado a AzureOpenAI


# =====

# Cargar variables de entorno

# =====

load_dotenv()


# =====

# Cliente Azure OpenAI

# =====

openai_client = AzureOpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_version=os.getenv("OPENAI_API_VERSION", "2024-08-01-preview")
)


# =====

# RAG Engine

# =====

class RAGEngine:

    def __init__(self, chroma_client: chromadb.Client, collection_name: str, openai_client:
AzureOpenAI):

        self.chroma_client = chroma_client

        self.collection = chroma_client.get_or_create_collection(collection_name)

        self.openai_client = openai_client

```

♦ Ingestar documentos con chunking jerárquico

```
def ingest_document(self, doc_id: str, text: str):
```

```
    large_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
```

```
    large_chunks = large_splitter.split_text(text)
```

```
    for section_id, large_chunk in enumerate(large_chunks):
```

```
        self.collection.add(
```

```
            ids=[f"{doc_id}_L{section_id}"],
```

```
            documents=[large_chunk],
```

```
            metadatas=[{
```

```
                "doc_id": doc_id,
```

```
                "section_id": section_id,
```

```
                "chunk_id": -1, # ✅ VALOR FIJO para chunks grandes
```

```
                "level": "large" # ✅ NIVEL CORRECTO
```

```
            ]
```

```
        )
```

```
    small_splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=50)
```

```
    small_chunks = small_splitter.split_text(large_chunk)
```

```
    for chunk_id, small_chunk in enumerate(small_chunks):
```

```
        self.collection.add(
```

```
            ids=[f"{doc_id}_L{section_id}_S{chunk_id}"],
```

```
            documents=[small_chunk],
```

```
            metadatas=[{
```

```
                "doc_id": doc_id,
```

```
                "section_id": section_id,
```

```
                "chunk_id": chunk_id, # ✅ VARIABLE del loop pequeño
```

```
                "level": "small" # ✅ NIVEL CORRECTO
```

```

    }}
)

```

♦ Recuperación jerárquica

```

def _build_context(self, query: str, top_k: int = 5) -> str:
    results = self.collection.query(query_texts=[query], n_results=top_k, where={"level": "small"})
    retrieved_chunks = []
    seen_large_sections = set()
    for doc, metadata in zip(results["documents"][0], results["metadatas"][0]):
        retrieved_chunks.append(doc)
        section_id = metadata["section_id"]
        doc_id = metadata["doc_id"]
        if (doc_id, section_id) not in seen_large_sections:
            parent = self.collection.query(
                query_texts=[query],
                n_results=1,
                where={"$and": [{"doc_id": doc_id}, {"section_id": section_id}, {"level": "large"}]}
            )
            if parent["documents"]:
                retrieved_chunks.append(parent["documents"][0][0])
                seen_large_sections.add((doc_id, section_id))
    return "\n\n".join(retrieved_chunks)

```

♦ Generación de respuesta

```

def generate_answer(self, query: str, top_k: int = 5) -> str:
    context = self._build_context(query, top_k=top_k)
    prompt = f"""

```

Usa el siguiente contexto para responder la pregunta.

Si no encuentras la respuesta en el contexto, responde "No lo sé".

Contexto:

{context}

Pregunta: {query}

Respuesta:

"""

```
        response = self.openai_client.chat.completions.create(
            model=os.getenv("OPENAI_MODEL_NAME", "gpt-4o"),
            messages=[
                {"role": "system", "content": "Eres un asistente experto en recuperación de
información."},
                {"role": "user", "content": prompt}
            ]
        )
        return response.choices[0].message.content
```

=====

Inicializar cliente Chroma y crear instancia singleton

=====

chroma_client = chromadb.Client()

rag_engine = RAGEngine(chroma_client, "docs_collection", openai_client)

Context_cache.py

cache/context_cache.py

import logging

import hashlib

from datetime import datetime, timedelta

```
logger = logging.getLogger(__name__)
```

```
class ContextCache:
```

```
    """Caché de contexto para documentos estáticos (alternativa simple a vLLM)"""
```

```
    def __init__(self):
```

```
        self.context_cache = {} # Almacena documentos por su hash
```

```
        self.access_times = {} # Seguimiento de últimos accesos para LRU
```

```
        logger.info("✅ Caché de contexto inicializada")
```

```
    def _generate_hash(self, content):
```

```
        """Genera un hash único para el contenido del documento"""
```

```
        return hashlib.md5(content.encode()).hexdigest()
```

```
    def add_document(self, document_content, metadata=None):
```

```
        """Añade un documento a la caché de contexto"""
```

```
        try:
```

```
            doc_hash = self._generate_hash(document_content)
```

```
            self.context_cache[doc_hash] = {
```

```
                "content": document_content,
```

```
                "metadata": metadata or {},
```

```
                "added_at": datetime.now()
```

```
            }
```

```
            # Actualizar tiempo de acceso
```

```
            self.access_times[doc_hash] = datetime.now()
```

```
            logger.debug(f"✅ Documento añadido a caché de contexto: {doc_hash[:8]}...")
```

```
return doc_hash
```

```
except Exception as e:
```

```
    logger.error(f"❌ Error añadiendo documento a caché: {str(e)}")
```

```
    return None
```

```
def get_document(self, document_content):
```

```
    """Recupera un documento de la caché si existe"""
```

```
    try:
```

```
        doc_hash = self._generate_hash(document_content)
```

```
        if doc_hash in self.context_cache:
```

```
            # Actualizar tiempo de acceso
```

```
            self.access_times[doc_hash] = datetime.now()
```

```
            logger.debug(f"✅ Documento encontrado en caché de contexto: {doc_hash[:8]}...")
```

```
            return self.context_cache[doc_hash]
```

```
        return None
```

```
except Exception as e:
```

```
    logger.error(f"❌ Error recuperando documento de caché: {str(e)}")
```

```
    return None
```

```
def search_in_cache(self, query, similarity_threshold=0.3): # Reducir threshold
```

```
    """
```

```
    Busca documentos en la caché que puedan responder la consulta
```

```
    """
```

```
    try:
```

```
        results = []
```

```

query_lower = query.lower()
query_words = query_lower.split()

for doc_hash, doc_data in self.context_cache.items():
    content = doc_data["content"].lower()

    # Búsqueda más flexible: contar coincidencias de palabras
    word_matches = sum(1 for word in query_words if word in content)
    similarity_score = word_matches / len(query_words) if query_words else 0

    # También verificar coincidencia parcial con el documento completo
    if similarity_score >= similarity_threshold or any(word in content for word in ["python",
"programación", "lenguaje"]):
        results.append({
            "document": doc_data["content"],
            "metadata": doc_data["metadata"],
            "similarity": similarity_score
        })

logger.debug(f"✅ Búsqueda en caché de contexto: {len(results)} resultados")
return results

except Exception as e:
    logger.error(f"❌ Error buscando en caché de contexto: {str(e)}")
    return []

def clear_old_entries(self, max_age_hours=24):
    """Limpia entradas antiguas de la caché"""
    try:

```

```

cutoff_time = datetime.now() - timedelta(hours=max_age_hours)

old_entries = [
    doc_hash for doc_hash, access_time in self.access_times.items()
    if access_time < cutoff_time
]

for doc_hash in old_entries:
    self.context_cache.pop(doc_hash, None)
    self.access_times.pop(doc_hash, None)

logger.info(f"🧹 Limpiadas {len(old_entries)} entradas antiguas de caché")

except Exception as e:
    logger.error(f"❌ Error limpiando caché: {str(e)}")

```

```

# Instancia singleton para ser importada
context_cache = ContextCache()

```

llms.py

```

# models/llms.py

import os

os.environ['NO_PROXY'] = 'recursoazureopenaimupi.openai.azure.com'

import logging

from openai import AzureOpenAI

from config.settings import AZURE_OPENAI_ENDPOINT, OPENAI_API_KEY, OPENAI_API_VERSION,
OPENAI_MODEL_NAME

logger = logging.getLogger(__name__)

```

```

class AzureOpenAIClient:
    """Cliente para interactuar con Azure OpenAI"""

    def __init__(self):
        self.client = AzureOpenAI(
            azure_endpoint=AZURE_OPENAI_ENDPOINT,
            api_key=OPENAI_API_KEY,
            api_version=OPENAI_API_VERSION,
        )
        self.model_name = OPENAI_MODEL_NAME
        logger.info(f"✅ Cliente de Azure OpenAI configurado con modelo: {self.model_name}")

    def generate_response(self, messages, temperature=0.1, max_tokens=2000):
        """Genera una respuesta usando el chat completion de Azure OpenAI"""
        try:
            response = self.client.chat.completions.create(
                model=self.model_name,
                messages=messages,
                temperature=temperature,
                max_tokens=max_tokens
            )
            return response.choices[0].message.content
        except Exception as e:
            logger.error(f"❌ Error en la generación con Azure OpenAI: {str(e)}")
            raise

# Instancia singleton para ser importada
azure_llm_client = AzureOpenAIClient()

```

redis_manager.py

```
# cache/redis_manager.py

import logging

import json

from datetime import datetime, timedelta

logger = logging.getLogger(__name__)

class MemoryCacheManager:

    """Gestor de caché de respuestas en memoria (reemplazo para Redis)"""

    def __init__(self):

        self.cache = {}

        logger.info("✅ Caché en memoria inicializada")

    def get_cached_response(self, query):

        """Obtiene una respuesta de la caché si existe y no ha expirado"""

        try:

            cache_key = f"response:{query}"

            if cache_key in self.cache:

                cached_data = self.cache[cache_key]

                # Verificar expiración

                if datetime.now() < cached_data["expiration"]:

                    logger.debug(f"✅ Respuesta encontrada en caché para: {query}")

                    return cached_data["response"]

            else:

                # Eliminar si expiró

                del self.cache[cache_key]
```

```

        return None

    except Exception as e:

        logger.error(f"❌ Error obteniendo de caché: {str(e)}")

        return None

def set_cached_response(self, query, response, expiration_hours=24):
    """Almacena una respuesta en la caché con tiempo de expiración"""
    try:
        cache_key = f"response:{query}"
        expiration_time = datetime.now() + timedelta(hours=expiration_hours)

        self.cache[cache_key] = {
            "response": response,
            "expiration": expiration_time
        }

        logger.debug(f"✅ Respuesta almacenada en caché para: {query}")
    except Exception as e:
        logger.error(f"❌ Error almacenando en caché: {str(e)}")

def clear_cache(self):
    """Limpia toda la caché (para desarrollo)"""
    try:
        self.cache.clear()

        logger.warning("🗑️ Caché en memoria limpiada")
    except Exception as e:
        logger.error(f"❌ Error limpiando caché: {str(e)}")

```

Instancia singleton para ser importada (usando el mismo nombre para no romper imports)

```
redis_cache = MemoryCacheManager()
```

settings.py

```
# config/settings.py
```

```
import os
```

```
from dotenv import load_dotenv
```

```
# Carga las variables del archivo .env
```

```
load_dotenv()
```

```
# Configuración de Azure OpenAI (tu LLM principal)
```

```
AZURE_OPENAI_ENDPOINT = os.getenv("AZURE_OPENAI_ENDPOINT",  
"https://recursoazureopenaimupi.openai.azure.com/")
```

```
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY",  
"35ftSYirsaOQsoEp07hTPUdlZ8h9xej48Lx9zg4ZsqaiozUnrEUuJQQJ99ALACHYHv6XJ3w3AAABACOGg  
VUd")
```

```
OPENAI_API_VERSION = os.getenv("OPENAI_API_VERSION", "2024-08-01-preview")
```

```
OPENAI_MODEL_NAME = os.getenv("OPENAI_MODEL_NAME", "gpt-4o")
```

```
# Configuración para Embeddings LOCAL Y GRATUITO
```

```
EMBEDDING_MODEL_NAME = os.getenv("EMBEDDING_MODEL_NAME", "all-MiniLM-L6-v2") #  
Modelo de SentenceTransformers
```

```
# Configuración de ChromaDB (base de datos vectorial LOCAL)
```

```
CHROMA_HOST = os.getenv("CHROMA_HOST", "localhost")
```

```
CHROMA_PORT = int(os.getenv("CHROMA_PORT", 8000))
```

```
CHROMA_COLLECTION_NAME = os.getenv("CHROMA_COLLECTION_NAME", "documentos")
```

```
# Configuración de Redis para caché de respuestas (opcional, puede ser local)
```

```
REDIS_HOST = os.getenv("REDIS_HOST", "localhost")
```

```

REDIS_PORT = int(os.getenv("REDIS_PORT", 6379))
REDIS_PASSWORD = os.getenv("REDIS_PASSWORD", None)

# Configuración de Langfuse para monitoreo (tiene plan gratis)
LANGFUSE_SECRET_KEY = os.getenv("LANGFUSE_SECRET_KEY", "")
LANGFUSE_PUBLIC_KEY = os.getenv("LANGFUSE_PUBLIC_KEY", "")
LANGFUSE_HOST = os.getenv("LANGFUSE_HOST", "https://cloud.langfuse.com")

# Configuración de Caché de Contexto (para documentos estáticos, alternativa simple a vLLM)
CONTEXT_CACHE_TYPE = os.getenv("CONTEXT_CACHE_TYPE", "memory") # 'memory' o 'disk'
CONTEXT_CACHE_PATH = os.getenv("CONTEXT_CACHE_PATH", "./.cache_context") # Si usa disk

# Valores por defecto para el orquestador
RAG_THRESHOLD = float(os.getenv("RAG_THRESHOLD", 0.7)) # Umbral de confianza para usar RAG

```

Chroma_client.py

```

# databases/chroma_client.py

import logging

import chromadb

from chromadb.config import Settings

from config.settings import CHROMA_HOST, CHROMA_PORT, CHROMA_COLLECTION_NAME

from models.embeddings import embedding_model

logger = logging.getLogger(__name__)

class ChromaClient:
    """Cliente para interactuar con ChromaDB"""

```

```

def __init__(self):
    try:
        # Configurar cliente de Chroma
        self.client = chromadb.HttpClient(
            host=CHROMA_HOST,
            port=CHROMA_PORT,
            settings=Settings(
                allow_reset=True,
                ssl_verify=False # <-- agrega esto para ignorar la verificación SSL (solo para pruebas
locales)
            )
        )

        # Crear o obtener la colección
        self.collection = self.client.get_or_create_collection(
            name=CHROMA_COLLECTION_NAME,
            metadata={"hnsw:space": "cosine"}
        )

        logger.info(f"✅ Cliente ChromaDB conectado a {CHROMA_HOST}:{CHROMA_PORT}")
        logger.info(f"✅ Colección '{CHROMA_COLLECTION_NAME}' lista")

    except Exception as e:
        logger.error(f"❌ Error conectando con ChromaDB: {str(e)}")
        raise

def add_documents(self, documents, ids=None, metadatas=None):
    """Añade documentos a la colección con sus embeddings"""
    if not documents:
        return

```

```

try:

    # Generar embeddings

    embeddings = embedding_model.get_embeddings(documents)

    # Si no se proporcionan IDs, generar automáticamente

    if ids is None:

        ids = [f"doc_{i}" for i in range(len(documents))]

    # Añadir a la colección

    self.collection.add(

        documents=documents,

        embeddings=embeddings,

        ids=ids,

        metadatas=metadatas

    )

    logger.info(f"✅ Añadidos {len(documents)} documentos a la colección")

except Exception as e:

    logger.error(f"❌ Error añadiendo documentos: {str(e)}")

    raise


def search(self, query_text, n_results=5):

    """Busca documentos similares a la consulta"""

    try:

        # Generar embedding de la consulta

        query_embedding = embedding_model.get_embeddings(query_text)[0]

```

```
# Realizar búsqueda

results = self.collection.query(

    query_embeddings=[query_embedding],

    n_results=n_results

)
```

```
return results
```

```
except Exception as e:
```

```
    logger.error(f"❌ Error en búsqueda: {str(e)}")
```

```
    return {'documents': [], 'ids': []}
```

```
def reset_collection(self):
```

```
    """Elimina todos los documentos de la colección (para desarrollo)"""
```

```
    try:
```

```
        # Método alternativo: obtener todos los IDs y eliminarlos
```

```
        existing_docs = self.collection.get()
```

```
        if existing_docs['ids']:
```

```
            self.collection.delete(ids=existing_docs['ids'])
```

```
            logger.warning(f"🔄 Colección limpiada: {len(existing_docs['ids'])} documentos eliminados")
```

```
        else:
```

```
            logger.info(f"📄 La colección ya está vacía")
```

```
except Exception as e:
```

```
    logger.error(f"❌ Error reseteando colección: {str(e)}")
```

```
    raise
```

```
# Instancia singleton para ser importada
```

```
chroma_client = ChromaClient()
```